

Deployment Document

Aurelia Dream Journal Application

1. Overview

Aurelia is a full-stack dream journal application developed using a layered n-tier architecture. The system allows users to create and manage dream entries, associate symbolic meanings with dreams, and generate poems inspired by dreams. The application demonstrates a complete service-oriented architecture built with assistance from generative AI tools.

The application includes the following layers:

- Client Layer (Web Interface)
- Service Layer (FastAPI REST API)
- Business Logic Layer
- Data Access Layer
- Database Layer (MySQL)

The goal of this project is to demonstrate the successful integration of these layers and to perform a complete system test verifying that all functionality works correctly.

2. System Architecture

The Aurelia system follows an n-tier architecture where each layer performs a specific role.

Client Layer (HTML / CSS / JavaScript)

↓

Service Layer (FastAPI REST API)

↓

Business Logic Layer (Service Classes)



Data Access Layer (Manager Classes)



Database Layer (MySQL)

Client Layer

The client layer is a web interface implemented using HTML, CSS, and JavaScript. It communicates with the backend through HTTP requests using the Fetch API.

Service Layer

The service layer is implemented using FastAPI and exposes REST endpoints for interacting with the database.

Business Logic Layer

This layer contains service classes that implement the business rules and logic required by the application.

Data Access Layer

Manager classes handle communication with the MySQL database and execute SQL queries.

Database Layer

The database is implemented using MySQL and stores all persistent application data.

3. Technologies Used

Technology	Purpose
Python	Core programming language

FastAPI	Service layer API framework
Uvicorn	ASGI server used to host FastAPI
MySQL	Relational database
HTML	Client interface structure
CSS	Client interface styling
JavaScript	Client-side API communication
VS Code	Development environment

4. Prerequisites

Before running the application, ensure the following software is installed:

Python

Version 3.10 or higher.

Verify installation:

```
python --version
```

MySQL Server

MySQL must be installed and running locally.

Verify installation:

```
mysql --version
```

Python Libraries

Install required dependencies:

```
pip install fastapi
```

```
pip install uvicorn
```

```
pip install mysql-connector-python
```

5. Downloading the Project

The project can be obtained from the GitHub repository.

Option 1: Clone Repository

```
git clone <repository_url>
```

Option 2: Download ZIP

1. Navigate to the GitHub repository page.
 2. Click **Code** → **Download ZIP**.
 3. Extract the ZIP file.
-

6. Database Setup

Start the MySQL server.

Create the application database:

```
CREATE DATABASE aurelia;
```

Switch to the database:

USE aurelia;

Create the required tables or import the schema provided in the repository.

Verify that the tables were created successfully:

SHOW TABLES;

Expected tables:

- users
- dreams
- symbols
- poems
- dreamsymbols

7. Running the Service Layer

Navigate to the project root directory.

Activate the Python virtual environment if one exists:

```
venv\Scripts\activate
```

Start the FastAPI server using Uvicorn:

```
uvicorn aurelia.app_api:app --reload
```

If successful, the server will start and display output similar to:

Uvicorn running on <http://127.0.0.1:8000>

The API documentation interface can be accessed at:

<http://127.0.0.1:8000/docs>

8. Running the Client Layer

The client interface is hosted using FastAPI's static file serving.

Open the application in a browser:

`http://127.0.0.1:8000/static/index.html`

The Aurelia Dream Journal interface should appear.

Users can navigate between:

- Users
 - Dreams
 - Symbols
 - Poems
 - Dream Symbols
-

9. Verifying Successful Deployment

The system is successfully deployed if:

- The FastAPI server runs without errors.
- The web client loads correctly in the browser.
- The client can communicate with the API.
- Database records are created and updated correctly.

Example verification test:

1. Create a new user through the client interface.
 2. Query the Users table in MySQL.
 3. Confirm that the record appears.
-

10. Client Interface Improvements

The client layer was enhanced to improve usability and accessibility.

Enhancements include:

- Dream journal themed user interface
- Navigation between entity sections
- Card-based display of results
- Removal of raw JSON output
- Human-readable results formatting

These improvements ensure the application is accessible to non-technical users.

11. Generative AI Assistance

Generative AI tools were used to assist with development of the application.

Examples of prompts used include:

Service Layer Prompt

Generate FastAPI CRUD endpoints for a MySQL database containing Users, Dreams, Symbols, and Poems tables.

Client Layer Prompt

Create a simple HTML and JavaScript web client that calls REST API endpoints using the fetch API.

UI Styling Prompt

Create a CSS theme with a soft dream journal aesthetic for a web interface.

12. AI Tool Effectiveness Analysis

Strengths

- Generated basic CRUD endpoint structures.
- Provided initial HTML client code.
- Assisted with styling and layout.

Weaknesses

- Some endpoints required manual corrections.
- JSON output required conversion to user-friendly displays.
- Database connections required debugging.

Improvements Made

The following changes were implemented manually:

- Fixed route parameter issues.
 - Added improved client navigation.
 - Created styled response display instead of raw JSON.
 - Implemented additional validation and UI improvements.
-

13. Conclusion

The Aurelia Dream Journal application successfully demonstrates a complete n-tier architecture built with the assistance of generative AI tools.

The system integrates:

- A MySQL database
- A FastAPI service layer
- A fully functional web client

All core CRUD operations were tested and verified across layers, confirming successful deployment and functionality.

Appendix – Repository Structure

AureliaDreamJournal

|

|— README.md

|— Deployment_System_Test.pdf

|

```
|— aurelia
|  |— app_api.py
|  |— services
|  |— managers
|  |— models
|
|— static
|  |— index.html
|  |— script.js
|  |— styles.css
|
|— test_frontend.py
```